
Endogenous Signals Fail to Allocate: Explicit Difficulty Supervision for Token-Level Adaptive Compute

Anonymous Author(s)

Affiliation

Address

email

Abstract

Token-level adaptive compute promises to make the transformer forward pass resource-aware, but most mechanisms rely on *endogenous* signals—attention entropy, router scores, confidence estimates—whose correlation with problem difficulty is assumed rather than tested. We test it directly. On arithmetic reasoning with graded difficulty, we show that endogenous difficulty signals fail in three distinct ways: an entropy-driven gate *collapses* to a constant (per-token entropy–difficulty correlation $r \approx -0.06$); a tag-initialized gate *inverts*, allocating more compute to easy problems than hard ones ($r = -0.59$) because the cross-entropy gradient is largest on memorizable tokens; and a learned difficulty head trained without supervision is *vacuous*, with unstable allocation across seeds. Explicit per-token difficulty supervision with a budget constraint fixes all three: gates order correctly (easy < medium < hard) in every seed with per-problem gate–difficulty $r = 0.95$, accuracy matches the full-compute baseline while cutting FLOPs per token by 42%, and it beats a static schedule at the same compute budget. On untagged inputs, a coarse difficulty proxy partially transfers ($r = 0.70$). When difficulty labels or proxies exist, supervised allocation is simple, reliable, and interpretable.

1 Introduction

Resource-aware agentic AI is mostly practiced *outside* the model: cascade queries across models of different sizes [Chen et al., 2023], truncate chains of thought, cap tool calls, budget retries. The forward pass itself remains resource-blind: a transformer [Vaswani et al., 2017] spends identical FLOPs on a trivial token and on the token that determines whether the answer is right. Yet difficulty is unevenly distributed across tokens—carrying a digit in multi-digit multiplication is hard; copying a “+” sign is not. An agent that must meter its compute should be able to allocate it per token.

Adaptive-compute methods show that large savings are available: early exiting [Teerapittayanon et al., 2017, Xin et al., 2020, Schuster et al., 2022], adaptive computation time [Graves, 2016], dynamic layer skipping [Wang et al., 2018, Elbayad et al., 2020], slimmable networks [Yu and Huang, 2019], and token-level Mixture-of-Depths (MoD) routing [Raposo et al., 2024] all reduce average compute per token. Nearly all of them share one assumption: that a signal the network already produces internally—attention entropy, router scores, exit confidence—correlates with how difficult a token is, so that end-to-end training on the task loss will shape that signal into a sensible allocation. This assumption is rarely tested directly. We test it, and it fails.

This paper is an empirical account of a four-iteration journey toward token-level adaptive compute on arithmetic reasoning with graded difficulty (Figure 1). Our testbed is a per-token continuous gate

35 $g \in [0, 1]$ that scales each token’s feed-forward (FFN) output at every layer, with FLOPs charged
 36 proportionally to the gate: a token gated at 0.2 costs a fifth of the FFN compute of a token gated at
 37 1.0. The question is what should drive the gate. Iteration 1 drives it with attention entropy, a label-
 38 free endogenous difficulty proxy: the gate collapses to a constant and entropy turns out not to encode
 39 task difficulty ($r \approx -0.06$ per token). Iteration 2 initializes the gating state from explicit difficulty
 40 tags: the gate now varies, but *inverts*—easy tokens receive more compute than hard ones ($r = -0.59$
 41 per problem)—because the cross-entropy gradient is largest on easy, memorizable tokens and end-
 42 to-end training follows the gradient, not the semantics. Iteration 3 replaces the entropy signal with a
 43 learned difficulty head but still trains it only on the task loss: the gate is nearly flat and its correlation
 44 with difficulty is unstable across seeds. Iteration 4 supervises the head explicitly with per-token
 45 difficulty targets plus a budget constraint, and it works.

46 Contributions.

- 47 • **A systematic diagnosis of endogenous difficulty signals.** Across three distinct instanti-
 48 ations (entropy-driven state, tag-initialized state, learned unsupervised head), endogenous
 49 training signals fail in three different ways—*collapse*, *inversion*, and *vacuity* (§4.2). A stan-
 50 dard MoD top- k router exhibits the same inversion, suggesting the pathology is a general
 51 property of endogenous training dynamics, not of our architecture.
- 52 • **A simple fix: explicit difficulty supervision with a budget constraint (§3).** A small per-
 53 layer difficulty head is trained with an MSE loss toward tag-derived per-token targets while
 54 a budget term pins the mean gate; a two-stage protocol with a nearly frozen body preserves
 55 task accuracy.
- 56 • **A controlled study** on tagged arithmetic (§4.3): gates order correctly (easy < medium <
 57 hard) in all 3 seeds with per-problem gate–difficulty $r = 0.95$; accuracy matches the full-
 58 compute baseline at 42% fewer FLOPs per token, and exceeds a static gate schedule at the
 59 same compute budget.
- 60 • **An analysis of generalization (§4.4, §4.5):** on untagged mixed-difficulty inputs a coarse
 61 digit-count proxy partially transfers (correct ordering, $r = 0.70$, small spread), and we
 62 identify when a static schedule suffices.

63 2 Related Work

64 **Adaptive compute.** Adaptive Computation Time [Graves, 2016] lets recurrent networks ponder
 65 per position, with a halting unit trained end-to-end. Early-exit methods attach classifiers to inter-
 66 mediate layers and stop when confident [Teerapittayanon et al., 2017, Xin et al., 2020, Elbayad
 67 et al., 2020, Schuster et al., 2022]. SkipNet [Wang et al., 2018] learns per-example layer skipping
 68 in convolutional networks, and universally slimmable networks [Yu and Huang, 2019] train a single
 69 network to run at multiple widths. Mixture-of-Depths [Raposo et al., 2024] routes tokens through
 70 or around transformer blocks with a learned top- k router, allocating a fixed compute budget per se-
 71 quence. All of these decide allocation from endogenous quantities—halting scores, exit confidence,
 72 router logits—trained on the task loss (sometimes with auxiliary balancing objectives). Our work is
 73 complementary: rather than proposing a new routing mechanism, we ask whether such endogenous
 74 signals actually track problem difficulty, and show that in our setting they do not unless supervised.
 75 This reframes the canonical method as well: MoD’s learned router exhibits the same inversion we
 76 diagnose (§4.5), allocating slightly more of its budget to easy tokens than to hard ones.

77 **Conditional computation and mixture-of-experts.** Sparsely-gated MoE [Shazeer et al., 2017]
 78 routes tokens to expert FFNs via a learned gating network, primarily to scale capacity at constant
 79 compute rather than to allocate by difficulty; load-balancing losses are needed to prevent collapse to
 80 a few experts—an early sign that end-to-end router dynamics do not self-organize into the intended
 81 allocation.

82 **Entropy and confidence as uncertainty proxies.** Attention entropy varies systematically across
 83 tokens and heads [Clark et al., 2019, Vig and Belinkov, 2019], and predictive confidence is widely
 84 used as a difficulty or calibration proxy [Guo et al., 2017], e.g. for early exits. We test attention
 85 entropy directly as a *task-difficulty* signal and find it nearly uncorrelated with ground-truth difficulty
 86 ($r \approx -0.06$ per token; §4.2).

87 **Resource-aware agents.** Orchestration-level resource awareness—model cascades and routing
 88 [Chen et al., 2023], token and tool budgets—treats each model call as fixed-cost. Token-level gat-
 89 ing makes the call itself cost-sensitive. Our results bear on when such metering is feasible: when
 90 difficulty labels or proxies exist (curriculum tags, verifier scores, metadata), supervised allocation is
 91 cheap and reliable; when only endogenous signals are available, allocation may be absent, inverted,
 92 or unstable.

93 3 Method

94 3.1 Architecture and per-token gating

95 Our backbone is an 11M-parameter decoder-only transformer: $d_{\text{model}} = 384$, 6 layers, 6 attention
 96 heads, $d_{\text{ff}} = 1536$, vocabulary 22, sequence length 256. Each layer computes

$$x_2 = \text{LN}(x_1 + \text{Attn}(x_1)), \quad x_3 = \text{LN}(x_2 + g_t \cdot \text{FFN}(x_2)), \quad (1)$$

97 where $g_t \in [0, 1]$ is a per-token, per-layer gate. Compute is charged proportionally: a position gated
 98 at g is billed g times the per-token FFN cost, while attention cost is always paid in full. A gate below
 99 1 thus saves FLOPs proportionally (about 42% of total per-token FLOPs at mean gate ≈ 0.5 , since
 100 the FFN dominates the layer budget).

101 In our final method, the gate is produced by a small per-layer *DifficultyHead*: a two-layer MLP
 102 (Linear(384→96), ReLU, Linear(96→1)) applied to the post-attention hidden state x_2 , whose
 103 scalar output s_t is squashed by a sigmoid, $g_t = \sigma(s_t)$. Earlier iterations used other drivers for
 104 the gate (attention entropy propagated through a low-dimensional recurrent “chemical state”, or a
 105 tag-initialized state); §4.2 describes them and their failure modes. All variants share Eq. (1) and the
 106 FLOPs accounting, so comparisons isolate the gating signal.

107 3.2 Why endogenous signals fail: three diagnostics

108 End-to-end training offers the gate only one channel of feedback: the task loss. We identify three
 109 ways this goes wrong, developed empirically in §4.2.

110 **Collapse.** If the driving signal carries no information about difficulty, the cheapest way to satisfy a
 111 sparsity penalty is a uniform gate. Attention entropy turns out to be such a signal: per-token entropy
 112 is nearly uncorrelated with problem difficulty ($r \approx -0.06$), and the gate collapses to a constant
 113 across difficulty bins.

114 **Inversion.** Giving the gating pathway a ground-truth initialization (difficulty-tag embeddings)
 115 makes the gate vary, but the gradient pushes it the wrong way: easy problems are memorizable,
 116 so their cross-entropy gradient is largest and training allocates them *more* compute ($r = -0.59$ per
 117 problem). The loss rewards reducing error where error is reducible, not where the problem is hard.

118 **Vacuity.** A learned difficulty head trained only through the task loss receives no explicit signal about
 119 what difficulty is: the gate stays nearly flat and its (weak) correlation with difficulty is unstable across
 120 seeds. Endogenous training provides neither the signal nor the incentive for reliable allocation; §3.3
 121 adds both.

122 3.3 Explicit difficulty supervision with a budget constraint

123 Our method trains the DifficultyHeads with explicit per-token targets derived from difficulty meta-
 124 data. Training is two-stage.

125 **Stage 1 (task training).** The transformer is trained conventionally for 5000 steps with the gate fixed
 126 at 1 everywhere, and checkpointed. This decouples representation learning from allocation learning.

127 **Stage 2 (allocation training).** Starting from the checkpoint, we attach the per-layer DifficultyHeads
 128 and train for 3000 steps with

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{mse}} \frac{1}{N} \sum_t (\sigma(s_t) - \tau_t)^2 + \lambda_{\text{budget}} (\bar{g} - 0.5)^2, \quad (2)$$

129 where τ_t is a per-position target gate and $\bar{g}$ the mean gate over the batch. On tagged data, targets
 130 come from the difficulty tag: easy $\rightarrow 0.2$, medium $\rightarrow 0.5$, hard $\rightarrow 0.8$, and non-problem positions

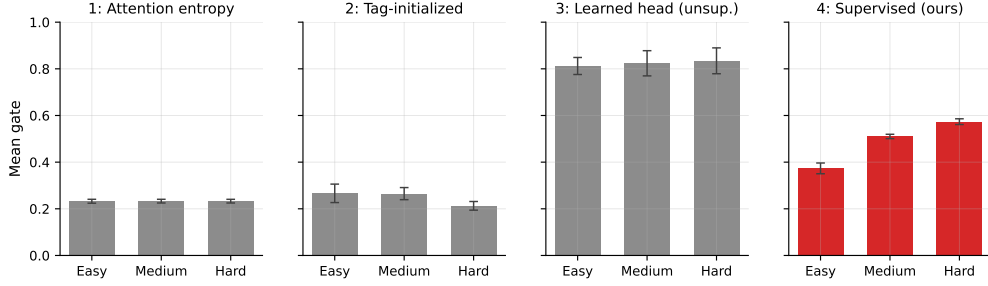


Figure 1: The four-iteration journey on tagged arithmetic: mean gate per difficulty bin (E/M/H) for each gating strategy. (1) Entropy-driven gate collapses to a constant. (2) Tag-initialized gate inverts—easy receives more compute than hard. (3) An unsupervised learned head is nearly flat. (4) Explicit difficulty supervision with a budget constraint produces the correct ordering with spread 0.20.

→ 0.5. We use $\lambda_{\text{mse}} = 5.0$ and $\lambda_{\text{budget}} = 0.5$; the budget term pins the mean gate near 0.5 (hence the total compute bill) while leaving the head free to redistribute compute *across* tokens. There is no sparsity penalty: the goal is allocation, not maximal savings. Stage 2 uses a two-group AdamW schedule: the body trains at a near-frozen learning rate 3×10^{-6} and the DifficultyHeads at 3×10^{-4} ; gradient clipping is 1.0 throughout. The low body learning rate preserves Stage-1 accuracy while the heads learn allocation; the cost is that the body can no longer adapt much in Stage 2, a confound we return to in §4.4 and §5.

4 Experiments

4.1 Setup

Tasks. We study arithmetic with graded difficulty in two regimes. In the *tagged* regime each problem is prefixed with a difficulty tag: [E] 1-digit addition/subtraction (~162 unique problems, memorizable), [M] 3-digit addition/subtraction (~1.6M unique), and [H] 4-digit $\times$ 3-digit multiplication (~9M unique). In the *mixed* regime there are no tags: operands have 1–4 digits and operators are $\{+, -, \times\}$; the difficulty bin is the digit count of the larger operand (1–4). Sequences pack multiple problems up to 256 tokens.

Evaluation. We use a frozen held-out set of 4000 problems per difficulty bin (seed 12345, identical across methods and seeds), report accuracy on answer tokens only, and run 3 seeds for every method. We report raw and balanced (macro over bins) accuracy, MFLOPs per token under the accounting of §3.1, the mean gate per difficulty bin, and the gate spread (hardest minus easiest bin).

Baselines. All baselines train end-to-end for 6000 steps: *Baseline* (standard transformer), *Chemical-off* (gating pathway frozen, gate = 1; an architecture-matched full-compute control), *Fixed-schedule* (gate fixed at 0.5 for every token; a static schedule at the same budget as ours), *MoD* (top- k token routing [Raposo et al., 2024] at matched capacity), *Chemical-entropy* (iteration 1: entropy-driven chemical-state gate with sparsity penalty), *Random-gate* (entropy replaced by uniform noise in the same range), and *Tag-only* (gate driven only by the difficulty-tag identity—a static oracle heuristic). Our method (*Difficulty-supervised*) uses the two-stage protocol of §3.3 (5000+3000 steps on tagged). All models share the 11M backbone, batch size 32, AdamW, and gradient clipping 1.0.

Calibration. Baselines train 6000 steps end-to-end while ours trains 5000+3000 with a nearly frozen body in Stage 2; comparisons below are “same task, similar total budget”, not a controlled iso-FLOPs training study.

4.2 Diagnosing the failure modes

Figure 1 summarizes the journey; Figure 2 quantifies the allocation quality per iteration.

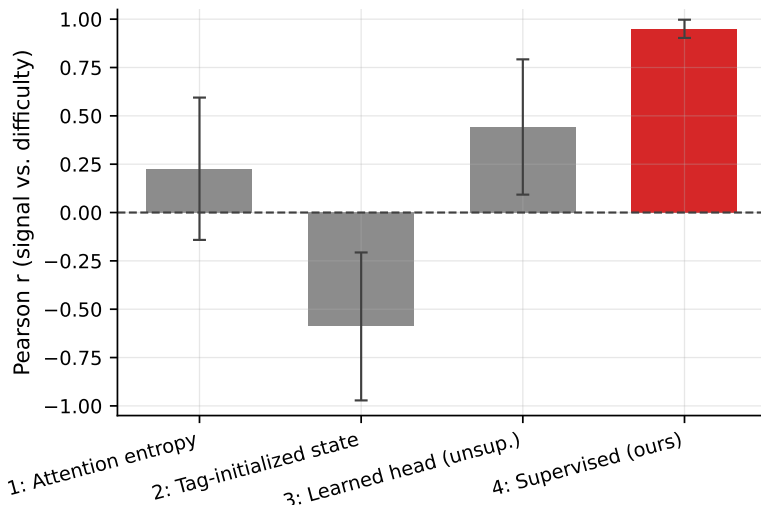


Figure 2: Per-problem correlation between mean gate and difficulty bin across the four iterations on tagged arithmetic (mean over 3 seeds). Endogenous variants fail by collapse ($r \approx 0$, iteration 1), inversion ($r = -0.59$, iteration 2), or instability (iteration 3); explicit supervision reaches $r = 0.95$.

Iteration 1: attention entropy collapses. A per-token low-dimensional chemical state, updated from position-normalized attention entropy and mapped to the gate, trained end-to-end with cross-entropy plus a sparsity penalty, converges to a constant gate: mean gates for E/M/H are 0.232/0.233/0.233 (spread ≈ 0.001). Accuracy is fine—71.5 $\pm$ 1.3% at 11.5 MFLOPs/token—so the model saves compute, but it does not allocate differentially. The reason is the signal, not the mechanism: the per-token Pearson correlation between attention entropy and difficulty is ≈ -0.06 (seed range +0.05 to -0.12). Attention entropy does not encode task difficulty in this setting.

Iteration 2: tag initialization inverts. Initializing the chemical state from the difficulty-tag embedding gives the pathway ground-truth information at the start of training. The gate does go live—but inverted: E/M/H gates are 0.266/0.265/0.213, and the per-problem correlation between mean gate and difficulty bin is -0.59 , negative in all 3 seeds. The cause is the training dynamics, not the initialization: the cross-entropy gradient is largest on easy (memorizable) tokens, so end-to-end training pushes compute where the loss is most reducible, not where the problem is hardest.

Iteration 3: a learned head without supervision is vacuous. Replacing the entropy pathway with the DifficultyHead of §3.1—two-stage protocol (Stage 1 full FFN for 3000 steps; Stage 2 head trained only by cross-entropy)—yields a nearly flat gate: E/M/H = 0.812/0.824/0.834, spread 0.02. The per-problem gate–difficulty correlation is unstable across seeds ($r = 0.65/0.64/0.04$). Without an explicit signal, there is no reliable allocation.

Iteration 4: explicit supervision works. With the loss of Eq. (2), gates order correctly in every seed; we develop this result next. The contrast across iterations is the point: the failure is not in any one mechanism but in the assumption that endogenous training will discover the allocation semantics on its own.

4.3 Tagged arithmetic: main result

Table 1 reports the main tagged results. Our method reaches 74.6 $\pm$ 2.7% accuracy (83.4 $\pm$ 2.1 balanced) at 19.3 MFLOPs/token—42% below the full-compute baseline’s 33.2. This statistically matches the baseline (75.0 $\pm$ 0.9) and the architecture-matched Chemical-off control (74.4 $\pm$ 0.7), while the allocation is correct: mean gates for E/M/H are 0.37/0.51/0.58, ordered in all 3 seeds, with spread 0.20 $\pm$ 0.03 and per-problem gate–difficulty $r = 0.95$ (0.97/0.98/0.90 per seed; Figure 3).

Two comparisons isolate the value of *adaptive* allocation. First, at the same compute budget, the Fixed-schedule baseline (gate 0.5 everywhere, 19.0 MFLOPs/token) reaches only 72.3 $\pm$ 2.7%:

Table 1: Tagged arithmetic: held-out accuracy (mean $\pm$ std over 3 seeds), balanced accuracy (macro over difficulty bins), compute per token, and gate spread (hardest minus easiest bin).

Method	Acc.	Balanced	MFLOPs/tok	Gate spread
Baseline	75.0 $\pm$ 0.9	83.8 $\pm$ 0.7	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Chemical-off	74.4 $\pm$ 0.7	83.4 $\pm$ 0.4	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Fixed-schedule (0.5)	72.3 $\pm$ 2.7	81.3 $\pm$ 2.3	19.0 $\pm$ 0.0	0.00 $\pm$ 0.00
MoD (top- k)	62.6 $\pm$ 4.8	72.3 $\pm$ 4.8	11.8 $\pm$ 0.0	-0.06 $\pm$ 0.01
Chemical (entropy)	71.5 $\pm$ 1.3	80.8 $\pm$ 1.3	11.5 $\pm$ 0.2	0.00 $\pm$ 0.00
Random-gate	70.7 $\pm$ 4.4	80.0 $\pm$ 4.2	11.2 $\pm$ 0.1	-0.00 $\pm$ 0.00
Tag-only	71.6 $\pm$ 3.1	80.9 $\pm$ 2.8	13.0 $\pm$ 0.4	-0.05 $\pm$ 0.09
Difficulty-supervised (ours)	74.6 $\pm$ 2.7	83.4 $\pm$ 2.1	19.3 $\pm$ 0.1	0.20 $\pm$ 0.03
Ours (Stage-1 = 3K)	66.9 $\pm$ 2.1	76.2 $\pm$ 2.4	19.6 $\pm$ 0.1	0.17 $\pm$ 0.03

Table 2: Mixed arithmetic: held-out accuracy (mean $\pm$ std over 3 seeds), balanced accuracy (macro over difficulty bins), compute per token, and gate spread (hardest minus easiest bin).

Method	Acc.	Balanced	MFLOPs/tok	Gate spread
Baseline	85.0 $\pm$ 1.7	90.6 $\pm$ 1.0	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Chemical-off	83.1 $\pm$ 2.7	89.0 $\pm$ 2.0	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Fixed-schedule (0.5)	86.6 $\pm$ 0.9	91.3 $\pm$ 0.7	19.0 $\pm$ 0.0	0.00 $\pm$ 0.00
MoD (top- k)	70.5 $\pm$ 2.9	78.5 $\pm$ 2.9	13.2 $\pm$ 0.0	-0.03 $\pm$ 0.05
Chemical (entropy)	84.1 $\pm$ 1.2	89.8 $\pm$ 0.8	12.8 $\pm$ 0.4	-0.00 $\pm$ 0.00
Random-gate	82.8 $\pm$ 4.4	88.8 $\pm$ 3.0	12.2 $\pm$ 0.8	0.00 $\pm$ 0.00
Difficulty-supervised (ours)	58.5 $\pm$ 3.4	69.9 $\pm$ 2.7	19.6 $\pm$ 0.1	0.08 $\pm$ 0.01

194 spending the same FLOPs uniformly rather than by difficulty costs ~ 2 points. Second, static or
 195 unsupervised alternatives at even lower budgets—Random-gate (70.7 $\pm$ 4.4 @ 11.2M), Tag-only
 196 (71.6 $\pm$ 3.1 @ 13.0M), Chemical-entropy (71.5 $\pm$ 1.3 @ 11.5M)—all trade accuracy away. Per-
 197 difficulty accuracies (Figure 4) show that our method preserves hard-bin accuracy while cutting
 198 compute on easy bins, which is where balanced accuracy is won.

199 4.4 Mixed arithmetic: partial generalization without tags

200 In the mixed regime there are no difficulty tags, so we supervise the head with a coarse proxy:
 201 digit count of the larger operand, mapping bins 1/2/3/4 to targets 0.2/0.4/0.6/0.8. (The mixed runs
 202 use a 3000-step Stage 1—only the tagged headline runs use the 5000-step variant, due to a time
 203 constraint.) Table 2 shows partial success. The allocation is correct in direction: mean gates per
 204 bin are 0.45/0.50/0.52/0.53, monotone increasing, with per-problem $r = 0.70$ —but the spread is
 205 small (0.08), and accuracy is 58.5 $\pm$ 3.4% (69.9 $\pm$ 2.7 balanced) at 19.6 MFLOPs/token, well below
 206 the 6000-step baselines (85.0 $\pm$ 1.7 for the standard baseline).

207 Context matters for interpreting this gap. Our mixed Stage-1-only checkpoint (3000 steps, full FFN,
 208 ungated) scores just 53.1% at 33.2 MFLOPs/token: the gated Stage-2 model *beats its own ungated*
 209 *starting point* by 5.4 points while using 41% fewer FLOPs. Because the body is nearly frozen in
 210 Stage 2 (learning rate 3×10^{-6}), most of the gap to the baselines is a training-budget artifact rather
 211 than a cost of gating. We therefore claim generalization only in a limited sense: a coarse difficulty
 212 proxy suffices to produce a correctly ordered, positively correlated allocation on untagged inputs,
 213 but the allocation is shallow and the accuracy deficit is real. §5 returns to what would be needed to
 214 close it.

215 4.5 Comparisons and ablations

216 **Static schedules.** The Fixed-schedule baseline answers the question “why not just shrink the FFN
 217 uniformly?” At matched budget on tagged data it loses ~ 2 accuracy points to our method (Table 1);
 218 on mixed data it is strong (86.6 $\pm$ 0.9 @ 19.0M), indicating that when per-token difficulty information
 219 is weak or unavailable, a static schedule is a surprisingly competitive default (see §5).

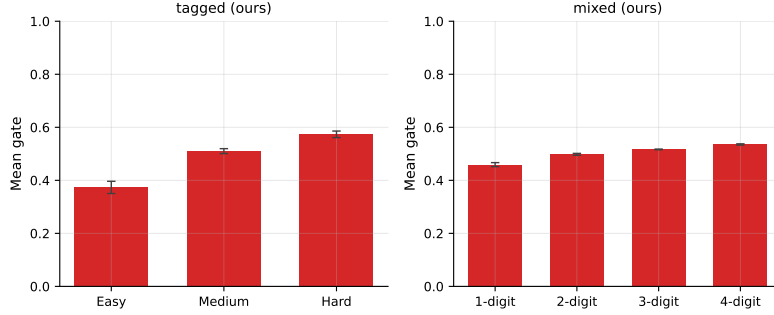


Figure 3: Mean gate per difficulty bin (E/M/H) on tagged arithmetic for our method and the failure-mode variants. Only explicit difficulty supervision produces the correct easy < medium < hard ordering consistently across seeds.

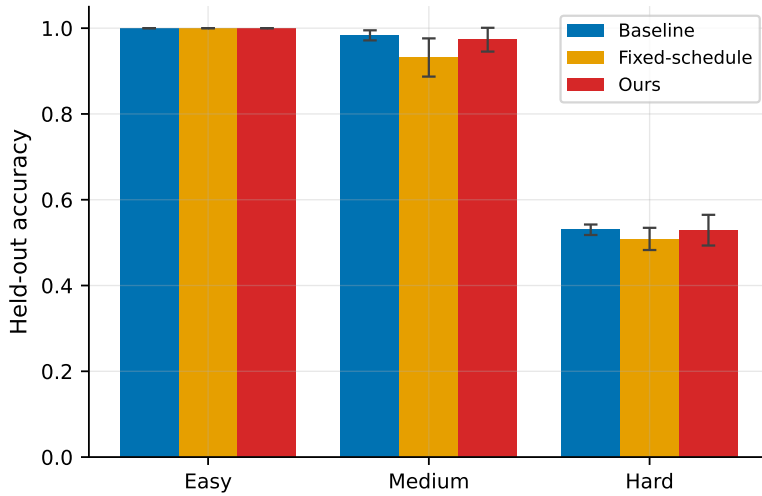


Figure 4: Per-difficulty held-out accuracy on tagged arithmetic. Our method keeps hard-bin accuracy close to the full-compute baseline while spending 42% fewer FLOPs per token; the fixed schedule at the same budget loses accuracy on the harder bins.

220 **MoD inverts too.** The inversion pathology is not specific to our architecture. A standard MoD top- k
 221 router [Raposo et al., 2024] allocates a slightly *larger* fraction of its compute budget to easy tokens
 222 than to hard ones (fraction of tokens processed, E/M/H = 0.135/0.109/0.093), and its accuracy
 223 suffers (62.6 ± 4.8 tagged, 70.5 ± 2.9 mixed). Endogenous training dynamics push compute toward
 224 reducible error regardless of the routing mechanism. MoD is thus not a counterexample to our
 225 diagnosis but its most prominent instance: the field’s canonical token-level allocator rests on the
 226 same untested assumption, and in our setting it fails the same way.

227 **Controls.** Random-gate (70.7 ± 4.4) and Tag-only (71.6 ± 3.1) show that neither gate variance alone
 228 nor a static oracle tag heuristic explains our result; Chemical-off (74.4 ± 0.7) confirms the gating
 229 pathway itself costs nothing when disabled.

230 **Stage-1 length.** Ablating Stage 1 from 5000 to 3000 steps (same protocol otherwise) drops tagged
 231 accuracy to 66.9 ± 2.1 @ 19.6 MFLOPs/token (Table 1, “Ours (Stage-1 = 3K)”: the body needs
 232 enough Stage-1 training before the allocation layer is added, consistent with the mixed-regime anal-
 233 ysis of §4.4.

234 **Frontier.** Figure 5 plots accuracy against MFLOPs per token for all methods on both regimes.
 235 On tagged data our method sits on the Pareto frontier at the high-accuracy end—the only adaptive
 236 method that matches full-compute accuracy at substantially reduced compute.

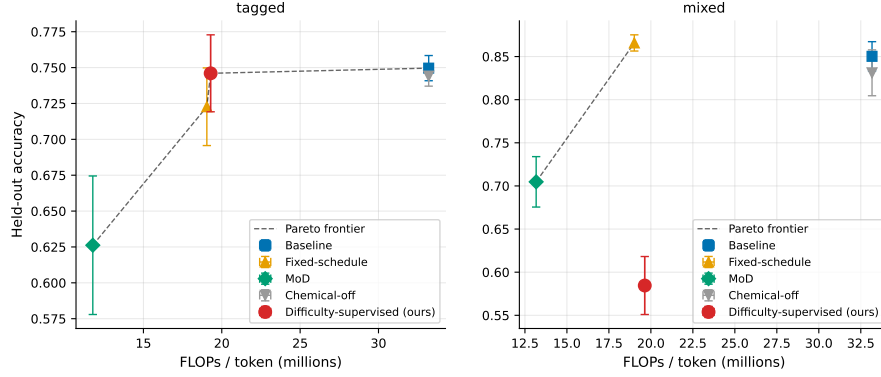


Figure 5: Accuracy–compute frontier on tagged and mixed arithmetic (3 seeds). On tagged data, difficulty-supervised gating matches full-compute accuracy at 42% fewer FLOPs per token and dominates the same-budget static schedule; on mixed data it improves over its own Stage-1 starting point but trails the longer-trained baselines.

237 5 Discussion

238 **Implications for resource-aware agents.** The practical lesson is about where difficulty knowl-
 239 edge comes from. Agentic pipelines often possess difficulty metadata that end-to-end training
 240 cannot discover on its own: curriculum tags in training corpora, verifier or reward-model scores,
 241 problem metadata, or cheap proxy features such as operand length. Our results show that spending
 242 this metadata on explicit per-token supervision yields an allocation that is reliable (correct order-
 243 ing in every seed), interpretable (gates can be read directly as a compute budget per token), and
 244 cheap (a two-layer head per transformer layer). Conversely, relying on signals the network already
 245 produces—entropy, router scores—risks allocation that is absent, inverted, or unstable, and the fail-
 246 ure can be silent: iteration 1 saved 65% of FLOPs with healthy accuracy while allocating nothing
 247 differentially. Notably, inversion is the more dangerous failure mode than collapse, since it actively
 248 misallocates while appearing to work.

249 **When static schedules suffice.** On mixed data the fixed schedule at the same budget beat every
 250 adaptive method (Table 2). Adaptive allocation pays when (i) difficulty varies substantially across
 251 tokens, (ii) a reliable difficulty signal or proxy exists, and (iii) the base model is already competent.
 252 When any of these fails—weak proxy (mixed regime), undertrained body (short Stage 1)—a uniform
 253 schedule is a strong baseline that adaptive methods must beat, not merely undercut on FLOPs.

254 **Limitations.** Our study is deliberately narrow: a single task family (arithmetic), a single 11M-
 255 parameter model, and difficulty defined by operand size; whether the same failure modes and fixes
 256 hold at scale and on natural language is open. FLOPs are an accounting of skippable compute: under
 257 dense execution wallclock time is unchanged, and realizing the savings requires sparse or gated
 258 kernels. The mixed-regime gap is partly a training-budget artifact, and our tagged comparisons
 259 are same-task, similar-budget rather than iso-FLOPs controlled. Finally, our targets are hand-set
 260 (0.2/0.5/0.8); learning the targets themselves, or deriving them from measured per-problem loss, is
 261 a natural next step.

262 6 Conclusion

263 We tested the assumption underlying most token-level adaptive-compute methods: that endogenous
 264 signals correlate with problem difficulty. On arithmetic with graded difficulty, they do not—failing
 265 by collapse, inversion, or vacuity across three distinct mechanisms, with a standard MoD router
 266 inverting as well. Explicit per-token difficulty supervision with a budget constraint fixes the allo-
 267 cation: correct ordering in every seed, per-problem $r = 0.95$, full-compute accuracy at 42% fewer
 268 FLOPs per token, and a win over the same-budget static schedule. For resource-aware agents, the
 269 message is conservative but useful: when difficulty labels or proxies exist, supervised token-level

allocation is a simple, reliable, and interpretable metering primitive; when they do not, endogenous signals should not be trusted to allocate on their own.

References

- Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance, 2023.
- Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does bert look at? an analysis of bert’s attention, 2019.
- Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer, 2020.
- Alex Graves. Adaptive computation time for recurrent neural networks, 2016.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.
- Tal Schuster, Adam Fisch, Jaakkola Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling, 2022.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.
- Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks, 2017.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.
- Jesse Vig and Yonatan Belinkov. Analyzing the structure of attention in a transformer language model, 2019.
- Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. Skipnet: Learning dynamic routing in convolutional networks, 2018.
- Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference, 2020.
- Jiahui Yu and Thomas S. Huang. Universally slimmable networks and improved training techniques, 2019.

A Training dynamics and fast validation

Figure 6 shows that the supervised allocation emerges quickly and is stable. As a fast-validation check, a run with only 500 Stage-2 steps already reached per-problem gate-difficulty $r = 0.988$ with E/M/H gates 0.43/0.54/0.60, indicating that the allocation signal is learned well before accuracy plateaus; the remaining Stage-2 steps mainly consolidate task accuracy under the budget constraint.

B Experimental details

Model and optimization. All models share the 11M-parameter decoder-only backbone of §3.1 ($d_{\text{model}} = 384$, 6 layers, 6 heads, $d_{\text{ff}} = 1536$, vocab 22, sequence length 256). We use batch size 32, AdamW, and gradient clipping at 1.0 throughout. Stage 2 uses two parameter groups: body at 3×10^{-6} , DifficultyHeads at 3×10^{-4} . The loss is Eq. (2) with $\lambda_{\text{mse}} = 5.0$, $\lambda_{\text{budget}} = 0.5$, budget target 0.5, and no sparsity penalty. Tagged targets are $E \rightarrow 0.2$, $M \rightarrow 0.5$, $H \rightarrow 0.8$ with non-problem positions at 0.5; mixed targets map digit bins 1/2/3/4 to 0.2/0.4/0.6/0.8.

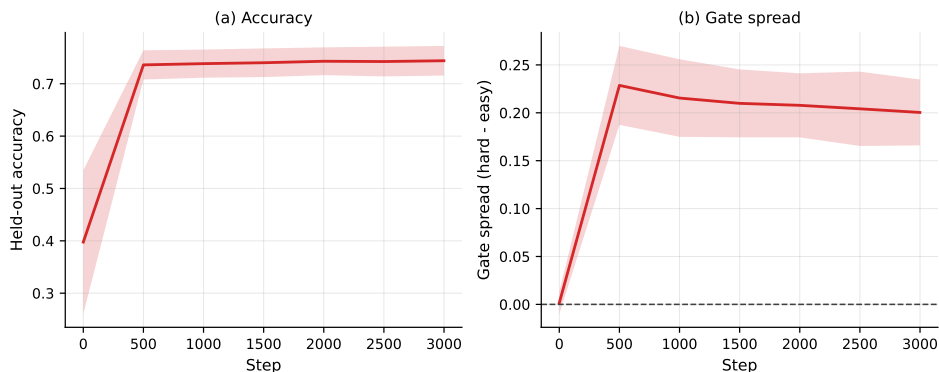


Figure 6: Training curves: held-out accuracy, MFLOPs per token, and per-bin mean gate over training for our method (tagged and mixed) and selected baselines. Stage 2 begins after the Stage-1 checkpoint; gates separate by difficulty within the first few hundred Stage-2 steps and then stabilize under the budget term.

Evaluation protocol. The held-out set contains 4000 problems per difficulty bin, generated once with seed 12345 and shared across all methods and seeds. Accuracy is computed on answer tokens only; balanced accuracy is the macro average over bins. Gate statistics are means over the layer-averaged gate at problem positions. Reported correlations are per-problem Pearson r between the mean gate of a problem’s positions and its difficulty bin, except where noted as per-token (iteration 1’s entropy analysis). All numbers are mean $\pm$ std over 3 seeds.

Per-difficulty details. Per-bin accuracies for all methods are visualized in Figure 4; per-bin gates in Figures 1 and 3. Gate spread is defined as the hardest-bin mean gate minus the easiest-bin mean gate.

Code. An anonymized implementation is available at <https://anonymous.4open.science/r/chemical-transformer-XXXX>.